

Departamento de Ingeniería en Computadoras
Facultad de Informática
Universidad Nacional del Comahue

Laboratorio en Programación Distribuida

Trabajo de Laboratorio Final

Programación Reactiva — Tateti Multijugador en Tiempo Real

Apellido y Nombre	Legajo	Email
Reibold, Sebastian Alejandro	FAI-3854	sebastian.reibold@est.fi.uncoma.edu.ar
Fernandez, Nicanor	FAI-3886	nicanor.fernandez@est.fi.uncoma.edu.ar
Fernandez Gramajo, Bautista	FAI-3874	bautista.fernandez@est.fi.uncoma.edu.ar

Índice

1. Introducción	2
2. Objetivos	3
2.1. Objetivo general	3
2.2. Objetivos específicos	3
3. Marco Teórico	4
3.1. Programación Reactiva	4
3.2. El Modelo Push vs. Pull	4
3.3. WebSockets	4
3.4. Socket.io	5
3.5. Programación Reactiva en el Frontend con React	6
4. Tecnologías Utilizadas	6
4.1. Backend	6
4.2. Frontend	6
5. Arquitectura de la Solución	7
5.1. Componentes del servidor	7
5.2. Componentes del cliente	7
5.3. Protocolo de eventos	8
6. Diseño e Implementación	9
7. Programación Reactiva en el Proyecto	9
7.1. El servidor como emisor de eventos (Push)	9
7.2. useEffect como suscriptor del stream de eventos	10
7.3. useState como estado derivado reactivo	10
7.4. Validación en el servidor: el cliente como vista, no como árbitro	10
7.5. Reconexión reactiva ante desconexión del rival	11
8. Problemas Encontrados y Soluciones Adoptadas	12
9. Instrucciones de Ejecución	13
10.Resultados Obtenidos	14
11.Conclusiones	15
12.Bibliografía	16

1. Introducción

El presente trabajo corresponde al Laboratorio Final de la materia *Laboratorio en Programación Distribuida*, cuyo eje central es el paradigma de **Programación Reactiva**. El objetivo del trabajo consiste en desarrollar una aplicación con comunicación bidireccional entre un cliente y un servidor, de forma tal que los cambios de estado en el servidor se vean reflejados automáticamente en el navegador del cliente sin necesidad de efectuar refrescos de página manuales, siguiendo el modelo en el que *el servidor empuja datos al cliente*.

La idea central que distingue a este tipo de arquitecturas del modelo cliente-servidor tradicional es precisamente esa inversión del flujo de información: en un sistema convencional, el cliente *pregunta* y el servidor *responde*; en un sistema reactivo, el servidor *notifica* y el cliente *reacciona*.

El nivel de complejidad elegido es el **Avanzado** (100 puntos), que corresponde al desarrollo de un Tateti (Tic-Tac-Toe) multijugador en tiempo real. Se trata del ejercicio más desafiante propuesto por el enunciado dado que requiere coordinar el estado compartido entre múltiples clientes simultáneamente, gestionar el ciclo de vida de las partidas y garantizar que los cambios de turno y el resultado final se propaguen en tiempo real a ambos jugadores.

La solución fue implementada utilizando **Node.js** con **TypeScript** en el servidor, **React** con **TypeScript** y **Vite** en el cliente, y **Socket.io** como biblioteca de comunicación en tiempo real.

2. Objetivos

2.1. Objetivo general

Comprender y aplicar los fundamentos de la Programación Reactiva mediante el desarrollo de una aplicación web multijugador en tiempo real, que implemente comunicación bidireccional entre cliente y servidor bajo el modelo de *push* de datos desde el servidor hacia el cliente.

2.2. Objetivos específicos

- Implementar una arquitectura cliente-servidor con comunicación en tiempo real utilizando el protocolo WebSocket a través de Socket.io.
- Construir un servidor en Node.js capaz de gestionar múltiples clientes de forma concurrente y mantener el estado compartido de las partidas activas.
- Desarrollar un cliente React con estado reactivo que se actualice automáticamente ante eventos emitidos por el servidor, sin necesidad de recarga de página.
- Aplicar el paradigma de Programación Reactiva tanto en la capa de comunicación (eventos de Socket.io) como en la capa de presentación (hooks de estado de React).
- Gestionar correctamente el ciclo de vida completo de una partida: emparejamiento de jugadores, validación de turnos en el servidor, detección de victoria o empate, y reconexión automática ante la desconexión del rival.
- Identificar y corregir problemas de seguridad, memory leaks y comportamientos inesperados del sistema durante el proceso de depuración.

3. Marco Teórico

3.1. Programación Reactiva

La Programación Reactiva es un paradigma de programación orientado a la propagación de cambios y al procesamiento asíncrono de flujos de datos (*streams*). Su premisa fundamental es que los sistemas deben estar diseñados para **reaccionar** ante eventos, en lugar de solicitar activamente el estado actual de un recurso de forma periódica (modelo *pull*).

El *Reactive Manifesto*, publicado en 2014 y firmado por más de veinte mil desarrolladores y organizaciones, define cuatro propiedades que deben exhibir los sistemas reactivos:

1. **Responsivos** (*Responsive*): el sistema responde a tiempo en todas las circunstancias. La capacidad de respuesta garantiza calidad de servicio y simplifica el manejo de errores.
2. **Resilientes** (*Resilient*): el sistema permanece funcional ante fallos. Los errores se contienen, se manejan de forma localizada y no se propagan al conjunto del sistema.
3. **Elásticos** (*Elastic*): el sistema mantiene su capacidad de respuesta ante variaciones en la carga de trabajo.
4. **Orientados a mensajes** (*Message-driven*): los componentes se comunican mediante mensajes asíncronos, lo que desacopla el remitente del receptor en tiempo y en espacio.

En el contexto de aplicaciones web, la programación reactiva se manifiesta típicamente a través de dos mecanismos complementarios: el **modelo de eventos** en el servidor (el servidor emite eventos ante cambios de estado) y la **gestión reactiva del estado** en el cliente (la interfaz de usuario se actualiza automáticamente cuando el estado cambia, sin intervención explícita del programador sobre el DOM).

3.2. El Modelo Push vs. Pull

En el modelo *pull* tradicional, el cliente es quien toma la iniciativa: pregunta periódicamente si hay información nueva (*polling*). Este enfoque tiene dos desventajas importantes: introduce latencia entre el momento en que ocurre un cambio en el servidor y el momento en que el cliente lo percibe, y genera tráfico innecesario cuando no hay cambios que reportar.

En el modelo *push*, en cambio, es el servidor quien toma la iniciativa: en cuanto ocurre un cambio relevante, *empuja* la nueva información hacia todos los clientes suscritos. La latencia se reduce drásticamente y el tráfico de red se genera únicamente cuando hay información nueva que transmitir.

El enunciado del laboratorio pone énfasis explícito en este punto: la idea central es que “*los cambios de estado en el Servidor se vean reflejados automáticamente en el navegador del cliente sin la necesidad de efectuar refrescos de la página actual (El servidor ‘empuja’ datos al cliente)*”. Esta descripción corresponde exactamente al modelo *push*, que es uno de los pilares de la programación reactiva.

3.3. WebSockets

HTTP, el protocolo sobre el que se construye la Web, fue diseñado para el modelo *request-response*: el cliente envía una solicitud, el servidor responde, y la conexión se cierra. Este modelo es eficiente para recursos estáticos pero impide que el servidor inicie la comunicación por sí mismo.

El protocolo **WebSocket** (RFC 6455, 2011) resuelve esta limitación estableciendo un canal de comunicación persistente, **full-duplex** y **bidireccional** entre el cliente y el servidor sobre una única conexión TCP. El establecimiento de la conexión comienza con un *handshake* HTTP estándar que incluye la cabecera **Upgrade: websocket**; si el servidor acepta, la conexión se “actualiza” al protocolo WebSocket y ambas partes quedan habilitadas para enviarse mensajes de forma independiente en cualquier momento, sin necesidad de que el otro extremo haya iniciado la comunicación.

Las ventajas del protocolo WebSocket para aplicaciones en tiempo real son:

- **Baja latencia:** no hay overhead de establecer una nueva conexión TCP por cada mensaje.
- **Comunicación bidireccional:** tanto el cliente como el servidor pueden iniciar mensajes.
- **Eficiencia:** las cabeceras de los mensajes WebSocket son mucho más pequeñas que las de HTTP.
- **Soporte nativo en navegadores:** la API **WebSocket** está disponible en todos los navegadores modernos.

3.4. Socket.io

Socket.io es una biblioteca de código abierto que abstrae la comunicación en tiempo real entre un servidor Node.js y clientes web. Aunque utiliza WebSocket como transporte preferente, también puede recurrir automáticamente a mecanismos de fallback (como *long polling*) cuando WebSocket no está disponible en el entorno del cliente, lo que garantiza compatibilidad universal.

Las características más relevantes de Socket.io para este proyecto son:

- **Sistema de eventos:** la comunicación se estructura como emisión y escucha de eventos nombrados, lo que resulta más expresivo y legible que un protocolo de texto plano.
- **Rooms** (salas): abstracciones que permiten agrupar sockets en canales lógicos. Un evento emitido hacia una sala llega a todos los sockets que la integran, lo que facilita el broadcast selectivo sin necesidad de iterar manualmente la lista de clientes.
- **Reconexión automática:** el cliente de Socket.io reintenta la conexión en caso de pérdida, con lógica de backoff exponencial.
- **Gestión del ciclo de vida:** eventos de conexión y desconexión (**connection**, **disconnect**) integrados nativamente.

3.5. Programación Reactiva en el Frontend con React

React es una biblioteca de JavaScript para la construcción de interfaces de usuario basada en el principio de **reactividad del estado**: cuando el estado de un componente cambia, React re-renderiza automáticamente la porción del árbol de componentes afectada. Esta propagación automática de cambios es una forma concreta de programación reactiva aplicada a la capa de presentación.

Los **hooks** de React relevantes para este proyecto son:

- **useState**: permite declarar piezas de estado dentro de un componente. Devuelve el valor actual y una función actualizadora. Cuando se invoca la función actualizadora, React programa un nuevo renderizado que refleja el estado actualizado.
- **useEffect**: permite ejecutar efectos secundarios (suscripción a eventos, llamadas a APIs, etc.) de forma sincronizada con el ciclo de vida del componente. El array de dependencias controla cuándo se re-ejecuta el efecto, y la función de limpieza (*cleanup*) se invoca al desmontar el componente.
- **useRef**: permite mantener una referencia mutable que no dispara re-renderizados, útil para acceder a valores en el interior de closures de efectos sin crear dependencias circulares.

La combinación de eventos de Socket.io y actualizaciones de estado con React implementa el patrón *Observer*, donde los eventos del servidor actualizan automáticamente la interfaz mediante cambios de estado.

4. Tecnologías Utilizadas

4.1. Backend

Node.js Entorno de ejecución de JavaScript basado en V8 y un modelo de E/S no bloqueante orientado a eventos. Se utilizó el módulo nativo **http** sin frameworks adicionales.

TypeScript Superconjunto tipado de JavaScript que permite detectar errores en compilación y mejorar la mantenibilidad del código. Se utilizó **ts-node** para la ejecución directa de archivos. Versión: 6.0.3.

Socket.io Biblioteca para comunicación en tiempo real mediante WebSockets. Se utilizó en servidor y cliente para sincronizar el estado del juego. Versión: 4.8.3.

4.2. Frontend

React 19 Biblioteca para la construcción de interfaces de usuario basada en componentes, hooks y renderizado reactivo.

Vite 8 Herramienta de desarrollo y compilación para aplicaciones web modernas, con recarga en caliente (HMR) y tiempos de inicio reducidos.

TypeScript Utilizado también en el cliente para aportar tipado estático y mayor seguridad en el manejo de datos.

Tailwind CSS v4 Framework CSS utilitario que permite definir estilos directamente desde clases y configurar temas mediante CSS con **@theme**.

5. Arquitectura de la Solución

La solución sigue una arquitectura **cliente-servidor con estado centralizado en el servidor** y comunicación bidireccional en tiempo real. El servidor es la única fuente de verdad (*single source of truth*) del estado del juego: todos los movimientos son validados en el servidor, y el estado actualizado es retransmitido a ambos jugadores simultáneamente. Los clientes son estrictamente presentacionales: no calculan el resultado del juego, solo muestran el estado que reciben y emiten eventos de usuario al servidor.

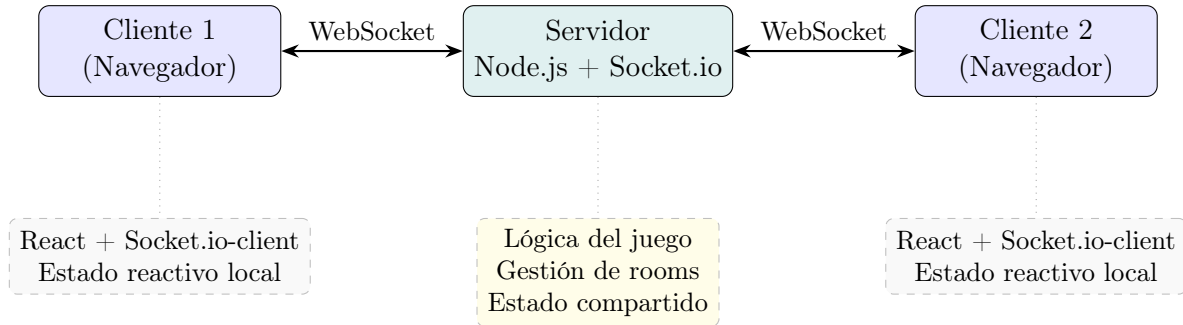


Figura 1: Arquitectura del sistema: clientes y servidor conectados mediante WebSocket

5.1. Componentes del servidor

El servidor está compuesto por tres módulos claramente delimitados:

- **Capa de tipos** (`server/types/index.ts`): define las interfaces y tipos de datos del dominio del juego (`Player`, `Cell`, `Board`, `GameResult`, `GameState`).
- **Lógica del juego** (`server/gameLogic.ts`): módulo de funciones puras sin dependencias externas ni conocimiento de sockets o red. Implementa la creación del estado inicial, la detección del ganador y la aplicación de movimientos.
- **Servidor principal** (`server/index.ts`): instancia el servidor HTTP y el servidor Socket.io, gestiona el ciclo de vida de las conexiones, administra las rooms y coordina el estado de las partidas activas mediante estructuras de datos en memoria.

5.2. Componentes del cliente

El cliente está organizado en capas con responsabilidades bien separadas:

- **Tipos del cliente** (`src/types/index.ts`): mirror de los tipos del servidor, con la adición del tipo `AppState` que modela los estados de la interfaz de usuario.
- **Instancia del socket** (`src/socket.ts`): singleton que crea la conexión con el servidor una única vez y la exporta.
- **Hook de juego** (`src/hooks/useGame.ts`): custom hook de React que encapsula toda la lógica de comunicación con el servidor y el estado local del juego.
- **Componentes de presentación**: `App.tsx` (contenedor principal), `StatusBar.tsx` (HUD con estado del juego), `Board.tsx` (tablero 3×3) y `Cell.tsx` (celda individual).

5.3. Protocolo de eventos

La comunicación entre cliente y servidor se basa íntegramente en eventos nominados de Socket.io, formando el siguiente protocolo:

Evento	Emisor	Receptor	Descripción
join	Cliente	Servidor	El jugador solicita unirse a una partida, enviando su nombre.
waiting	Servidor	Cliente 1	Notifica al primer jugador que debe esperar rival.
start	Servidor	Ambos	Inicia la partida. Lleva el estado inicial, el símbolo asignado y el nombre del rival.
move	Cliente	Servidor	El jugador informa el índice de la celda que desea ocupar (0-8).
update	Servidor	Ambos	Retransmite el nuevo estado del juego tras un movimiento válido.
opponent_left	Servidor	Rival	Notifica que el otro jugador abandonó la partida.
disconnect	Socket.io	Servidor	Evento interno de cierre de conexión.

Cuadro 1: Protocolo de eventos de la aplicación

6. Diseño e Implementación

El sistema se estructura en módulos con responsabilidades claramente delimitadas. Los tipos del dominio (`Cell`, `Board`, `GameState`) se definen en `types/index.ts` y establecen un contrato explícito sobre la forma del estado compartido entre servidor y cliente. El módulo `gameLogic.ts` encapsula toda la lógica del dominio en funciones puras sin dependencias externas: `checkWinner` recorre las ocho combinaciones ganadoras del Tateti con un bucle `while` que termina en cuanto se detecta un resultado, y `makeMove` aplica la inmutabilidad retornando un nuevo objeto de estado sin mutar el anterior. Esta separación permite razonar sobre la corrección del juego de forma completamente independiente a la capa de red.

El servidor principal gestiona el estado de todas las partidas activas en cuatro mapas en memoria: `rooms` (estado de juego por sala), `playerRoom` (sala por socket), `playerSymbol` (símbolo asignado por socket) y `playerName` (nombre por socket). El handler del evento `join` implementa el emparejamiento automático: si existe una sala con un solo jugador en espera se une a ella, de lo contrario crea una sala nueva. Al completarse la sala, el servidor emite `start` a ambos sockets de forma simultánea con el estado inicial, el símbolo asignado y el nombre del rival. El handler de `move` valida que el evento provenga del socket cuyo símbolo coincide con el turno actual antes de aplicar el movimiento y retransmitir el nuevo estado; si la validación falla, el evento se descarta silenciosamente sin notificar al cliente.

En el cliente, la conexión se establece una única vez como singleton en `socket.ts` y se comparte entre todos los módulos, evitando múltiples conexiones TCP simultáneas. El custom hook `useGame` concentra toda la lógica reactiva: mediante un único `useEffect` sin dependencias suscribe manejadores a los cuatro eventos del servidor (`waiting`, `start`, `update`, `opponent_left`) y los desregistra en la función de cleanup al desmontar el componente. Cada evento recibido produce actualizaciones de estado local mediante `useState`, manteniendo la interfaz sincronizada con el estado canónico del servidor. Los componentes de presentación (`App`, `Board`, `Cell`, `StatusBar`) son puramente declarativos: reciben props del hook y renderizan JSX sin lógica de dominio ni comunicación directa con el socket.

7. Programación Reactiva en el Proyecto

Esta sección analiza explícitamente cómo los conceptos de programación reactiva se materializan en los distintos niveles del sistema, qué componentes los implementan y qué beneficios concretos aportan.

7.1. El servidor como emisor de eventos (Push)

El núcleo del paradigma reactivo en este proyecto reside en el servidor. En lugar de responder únicamente a solicitudes del cliente, el servidor **emite eventos proactivamente** ante cambios de estado:

- Cuando el segundo jugador se une a una sala, el servidor emite `start` a **ambos jugadores simultáneamente**, sin que ninguno de ellos lo haya pedido explícitamente.
- Cuando un jugador realiza un movimiento válido, el servidor aplica el cambio, calcula el nuevo estado y emite `update` a **toda la sala** (ambos jugadores) de forma atómica con `io.to(roomId).emit(...)`.
- Cuando un jugador se desconecta, el servidor emite `opponent_left` al jugador restante para que pueda reaccionar.

Este patrón elimina la necesidad de cualquier forma de *polling* en el cliente: no hay ningún temporizador que pregunte periódicamente al servidor si el estado cambió. Los cambios se propagan instantáneamente en cuanto ocurren.



Figura 2: Flujo reactivo: el servidor empuja el nuevo estado a ambos jugadores tras un movimiento

7.2. `useEffect` como suscriptor del stream de eventos

En el cliente, el hook `useEffect` actúa como el mecanismo de suscripción al flujo de eventos del servidor. Cuando el componente se monta, el efecto registra los *listeners* para los eventos `waiting`, `start`, `update` y `opponent_left`. Cuando el componente se desmonta, el `cleanup` elimina esos listeners.

Esta estructura es funcionalmente equivalente al patrón **Observer** (o al operador `subscribe` de RxJS): el socket actúa como `Observable`, y el cuerpo de `useEffect` define las acciones a ejecutar ante cada evento (`next`). La función de `cleanup` devuelta por `useEffect` es el equivalente a `unsubscribe`.

7.3. `useState` como estado derivado reactivo

Las tres llamadas a `useState` dentro de `useGame` son el mecanismo por el que los cambios del servidor se propagan automáticamente a la interfaz de usuario:

1. El servidor emite `update` con el nuevo estado del juego.
2. El listener del cliente invoca `setGameState(state)`.
3. React detecta que el estado cambió y re-renderiza automáticamente todos los componentes que dependen de `gameState`.
4. `Board` re-renderiza mostrando la celda recién ocupada con la animación `piece-pop`.
5. `StatusBar` re-renderiza mostrando el nuevo turno o el resultado de la partida.

En ningún punto el código del programador manipula el DOM directamente ni programa un re-render manualmente. La propagación de cambios es completamente automática: se describe el *qué* (el estado), no el *cómo* (la mutación del DOM). Esto es la esencia de la programación reactiva declarativa.

7.4. Validación en el servidor: el cliente como vista, no como árbitro

Una consecuencia importante del modelo reactivo adoptado es que **el cliente no es el árbitro del juego**: no calcula si un movimiento es válido, no detecta el ganador, no alterna los turnos. El cliente envía la intención del usuario (`socket.emit('move', index)`) y el servidor decide si esa intención es válida, aplica el cambio y retransmite el estado canónico.

Esta separación tiene implicaciones directas en seguridad: un usuario malintencionado podría abrir la consola del navegador y emitir eventos de socket manualmente, pero el servidor rechazará cualquier movimiento que no corresponda al turno de ese socket específico, ignorándolo en silencio. La integridad del juego está garantizada por el servidor, no por el cliente.

7.5. Reconexión reactiva ante desconexión del rival

Cuando el rival se desconecta, el servidor emite `opponent_left` al jugador restante. El cliente, al recibir este evento, reacciona de dos formas: limpia su estado local (resetea `gameState`, `mySymbol` y `opponentName`) y, sin recargar la página, emite automáticamente un nuevo `join` para volver a la cola de emparejamiento.

```
1 socket.on('opponent_left', () => {
2   setGameState(null)
3   setMySymbol(null)
4   setOpponentName(null)
5   setAppState('waiting')           // interfaz actualizada
6   socket.emit('join', { name: myNameRef.current }) // reencolar sin
7   recargar
8 })
```

Listing 1: Reconexion automatica reactiva al evento `opponent_left`

Esta secuencia ejemplifica el modelo reactivo en acción: un evento del servidor desencadena una cadena de actualizaciones de estado que se propagan automáticamente a la interfaz, y simultáneamente disparan una nueva acción sobre el servidor, todo sin intervención explícita del usuario.

8. Problemas Encontrados y Soluciones Adoptadas

Durante la revisión del código se detectaron y corrigieron cinco problemas. El servidor carecía de validación de turno, lo que permitía que cualquier cliente emitiera movimientos fuera de su turno; se solucionó incorporando el mapa `playerSymbol` y verificando el turno al inicio del handler `move`. Al desconectarse un jugador, el handler `disconnect` no limpiaba las entradas del rival en los mapas de estado, causando crecimiento indefinido de memoria; la solución fue iterar los sockets restantes de la sala antes de eliminarla. Cuando el rival se desconectaba, el jugador quedaba bloqueado porque el evento `join` nunca se re-emitía; se resolvió agregando un `rejoin` automático en el handler de `opponent_left`, usando `useRef` para acceder al nombre del jugador dentro del closure sin crear dependencias en el `useEffect`. El modo `React.StrictMode` provoca una doble ejecución de efectos en desarrollo, generando dos eventos `join` desde el mismo socket; se agregó la guarda `if (playerRoom.has(socket.id)) return` al inicio del handler. Finalmente, el componente `Board` se renderizaba incondicionalmente, mostrando su contenedor vacío durante la espera; se condicionó el renderizado a los estados `playing` y `finished`.

9. Instrucciones de Ejecución

El proyecto requiere Node.js instalado en el sistema. Para ejecutar la aplicación se necesitan dos terminales simultáneas:

```
1 # Terminal 1 - Servidor (puerto 3000)
2 cd tateti-realtime/server
3 npm install
4 npm run dev
5
6 # Terminal 2 - Cliente (puerto 5173)
7 cd tateti-realtime/client
8 npm install
9 npm run dev
```

Listing 2: Ejecucion del servidor y del cliente

Una vez iniciados ambos procesos, abrir dos pestañas del navegador en <http://localhost:5173>. Cada pestaña representa un jugador diferente. Ingresar un nombre en cada una y presionar “JUGAR”; el emparejamiento ocurre automáticamente en cuanto las dos pestañas han enviado el evento `join`.

La variable de entorno `VITE_SERVER_URL=http://localhost:3000` en el archivo `client/.env` define la URL del servidor. Para ejecutar el servidor en un puerto diferente, actualizar esa variable y el valor de `PORT` en `server/index.ts`.

10. Resultados Obtenidos

Los resultados concretos son:

- **Comunicación bidireccional en tiempo real:** el servidor empuja el estado del juego a ambos clientes en cuanto ocurre cualquier cambio, sin ninguna forma de polling. La latencia percibida es la del round-trip de la red local.
- **Emparejamiento automático:** el servidor gestiona una cola de salas y asigna jugadores automáticamente sin intervención del usuario.
- **Flujo completo de la partida:** desde el ingreso del nombre hasta el resultado final (victoria, derrota o empate), incluyendo la indicación visual del turno activo en tiempo real.
- **Reconexión automática:** si un jugador abandona la partida, el rival regresa automáticamente a la cola de emparejamiento sin necesidad de recargar la página.
- **Seguridad básica en el servidor:** la validación de turnos en el servidor garantiza que ningún cliente pueda alterar el estado del juego fuera de su turno, independientemente de lo que emita por socket.
- **Interfaz de usuario reactiva y visualmente polida:** el diseño Synthwave Pixel Art con animaciones CSS proporciona una experiencia de usuario fluida y con identidad visual definida.
- **Código sin bugs conocidos:** los cinco problemas detectados durante la revisión fueron corregidos y verificados.

11. Conclusiones

El desarrollo de este trabajo final permitió comprender cómo el paradigma de Programación Reactiva puede aplicarse en una aplicación web real. Esto se logró mediante dos componentes principales: la comunicación en tiempo real entre servidor y clientes usando Socket.io, y la actualización automática de la interfaz mediante los hooks de React.

La principal diferencia con una aplicación web tradicional es que el servidor no espera constantemente consultas de los clientes, sino que les informa automáticamente cuando ocurre un cambio. De esta forma, los clientes reaccionan a los eventos recibidos y actualizan su estado en tiempo real. Esto evita el uso de polling, reduce los tiempos de espera y permite que todos los usuarios vean la misma información de manera sincronizada.

La utilización de Socket.io resultó una buena elección, ya que simplifica la comunicación en tiempo real y ofrece herramientas como las rooms para enviar eventos únicamente a los clientes necesarios. Además, su modelo basado en eventos se integra de forma natural con React. Por otro lado, mantener el estado principal del juego en el servidor y separar la lógica de negocio de la comunicación facilitó el desarrollo, el mantenimiento y la depuración de la aplicación.

En conclusión, la Programación Reactiva propone una forma diferente de construir aplicaciones: en lugar de responder continuamente a solicitudes, los sistemas reaccionan y propagan cambios cuando estos ocurren. Este trabajo permitió comprobar que este enfoque es práctico, eficiente y perfectamente aplicable al desarrollo de aplicaciones web modernas.

12. Bibliografía

Referencias

- [1] Bonér, J., Farley, D., Kuhn, R. y Thompson, M. (2014). *The Reactive Manifesto* (v2.0). Recuperado de <https://www.reactivemanifesto.org>
- [2] Fette, I. y Melnikov, A. (2011). *The WebSocket Protocol* (RFC 6455). Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/rfc6455>
- [3] Socket.io Contributors. (2024). *Socket.io Documentation*. Recuperado de <https://socket.io/docs/v4>
- [4] Meta Platforms, Inc. (2024). *React Documentation — Hooks Reference*. Recuperado de <https://react.dev/reference/react>
- [5] OpenJS Foundation. (2024). *Node.js Documentation*. Recuperado de <https://nodejs.org/en/docs>
- [6] Vite Contributors. (2024). *Vite Documentation*. Recuperado de <https://vite.dev>
- [7] Tailwind Labs. (2024). *Tailwind CSS v4 Documentation*. Recuperado de <https://tailwindcss.com/docs>
- [8] Coulouris, G., Dollimore, J., Kindberg, T. y Blair, G. (2011). *Distributed Systems: Concepts and Design* (5th ed.). Addison-Wesley. ISBN: 978-0132143011.
- [9] Tanenbaum, A. S. y Van Steen, M. (2006). *Distributed Systems: Principles and Paradigms* (2nd ed.). Prentice Hall. ISBN: 978-0132392273.